

ISPTEC
Instituto Superior Politécnico de Tecnologias e Ciências

ESTRUTURA DE DADOS I

Autor: Sílvia António
Área: DET

O QUE VAMOS ESTUDAR ?

I. Revisão da linguagem C
II. Ponteiros e Tipos Abstractos de Dados
III. Listas
IV. Pilhas
V. Filas
VI. Arvores

Autor: Sílvia António

Ponteiros e Tipos Abstractos de Dados

- **Ponteiros**
- Alocação Dinâmica de memória
- Tipos Abstractos de Dados

Autor: Sílvia António
Ano Académico: 2026/2027

Objectivos

- Explicar o que é um endereço de memória e o que é um ponteiro;
- Declarar e usar ponteiros com os operadores & e *;
- Usar ponteiros para alterar variáveis dentro de funções;
- Relacionar ponteiros e arrays.

Autor: Sílvia António
Ano Académico: 2026/2027

O que são ponteiros?

- Toda a variável tem um **valor** e um **endereço** de memória.
- Um ponteiro é uma variável que guarda o **endereço** de outra variável.

Autor: Sílvia António
Ano Académico: 2026/2027

Endereços em C

- Para qualquer variável **x,** o seu endereço é indicado por **&x.**
- Por exemplo, ao usarmos a função **scanf("%d", &x)** o segundo argumento é o endereço da variável.

Exemplo:

```c
#include <stdio.h>
main()
{
    int idade = 20;

   //Imprime o valor da variável
   printf("A tua idade: %d\n", idade);

   // Imprime o endereço da variável
   printf("Endereço da variável idade: %p", &idade);

 }
```

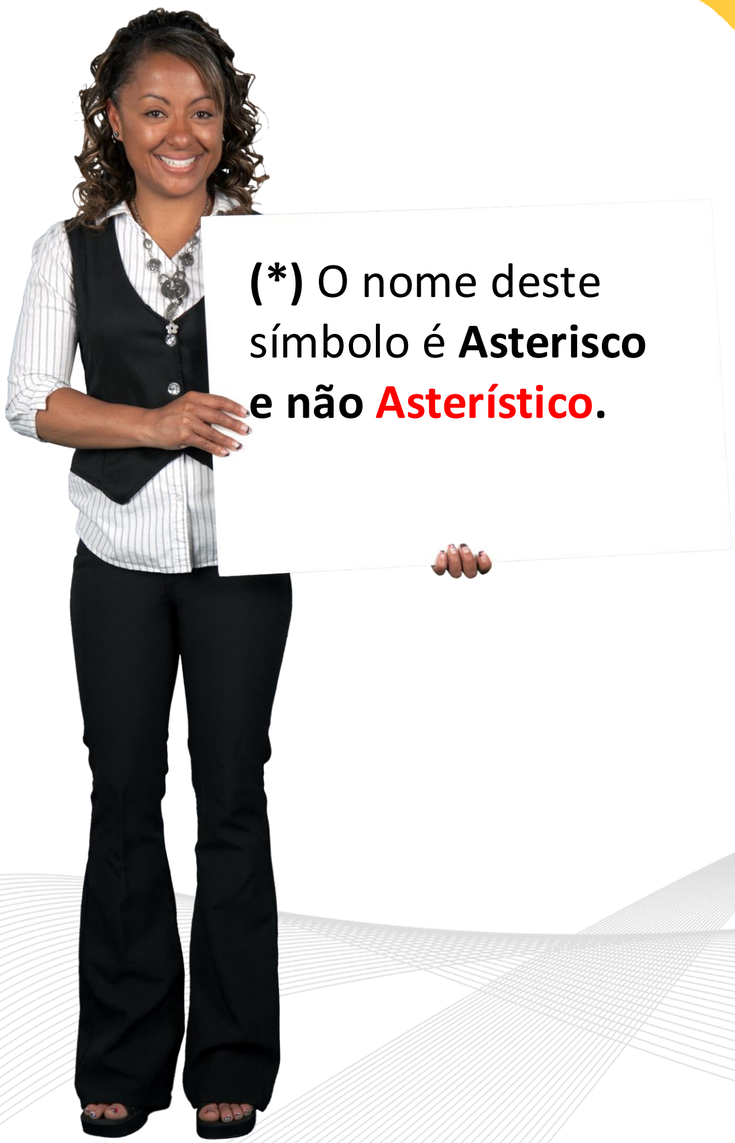
Autor: Sílvia António
Ano Académico: 2026/2027

Declaração de ponteiros

tipo * nome_da_variável;

Exemplo:

```
int * idade;  //Ponteiro do tipo inteiro
int * x, k;      //x é ponteiro e k é uma variável normal
char * nome;  //Ponteiro do tipo char
int * p = NULL;   //p é um ponteiro nulo
```

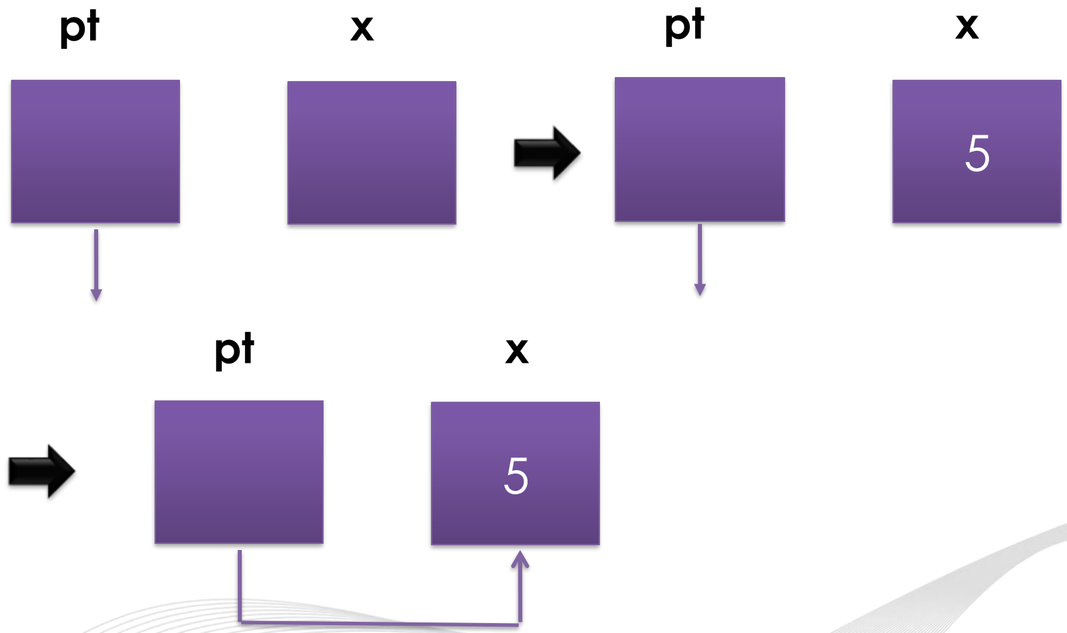


Autor: Sílvia António
Ano Académico: 2026/2027

Atribuição de ponteiros

Exemplo:

```
int x, * pt;
x = 5;
pt = &x;
```



Autor: Sílvia António
Ano Académico: 2026/2027

Valor da variável apontada

O operador asterisco (*) também é usado para aceder ao valor da variável apontada por um ponteiro.

Exemplo1:

```
int x, *pt;
x = 31;
pt =&x;
printf("%d", *pt);  //Imprime 31
*pt = 28;
printf("%d", x);  //Imprime 28
```

Autor: Sílvia António
Ano Académico: 2026/2027

Valor da variável apontada

Exemplo2:

```
int* pc, c, d;
c = 6;
d = -8;

pc = &c;
printf("%d", *pc); // Imprime 6
pc = &d;
printf("%d", *pc); // Imprime -8
```

Operações aritméticas

- **Incremento** – Desloca o ponteiro para o endereço imediatamente seguinte do mesmo tipo.
- **Decremento** - Desloca o ponteiro para o endereço imediatamente anterior do mesmo tipo.

Exercícios

1. Faça um programa que peça do utilizador o nome, a idade e o peso. Imprima os valores inseridos, bem como o endereço das variáveis usadas.
2. Sem usar o compilador, analise o código abaixo e indique o valor de cada variável no final de toda a execução.

```
int i=7, j=5;
int * p, *q;
p = &i;
q = &i;
*q =5;
(*q) ++;
```

Autor: Sílvia António
Ano Académico: 2026/2027

Ponteiros e vectores

- O nome de um vector é um ponteiro para o se primeiro elemento, ou seja, guarda o endereço do primeiro elemento.

Verifique o que é impresso pelas instruções abaixo:

```
int v[4];
printf("%p", v);
printf("%p", &v);
printf("%p", &v[0]);
printf("%p", &v[1]);
printf("%p", v+1);
```

Autor: Sílvia António
Ano Académico: 2026/2027

Ponteiros e vectores - Exemplo

```
#include <stdio.h>
int main() {

int i, num[6], soma = 0;

printf("Insira 6 números: ");

for(i = 0; i < 6; ++i) {
      scanf("%d", num+i); // Equivalente a scanf("%d", &num[i]);

      soma += *(num+i);  // Equivalente a soma += num[i]
}
printf("Soma = %d", soma);

return 0;
}
```

Autor: Sílvia António
Ano Académico: 2026/2027

Ponteiros e vectores

- O valor do endereço de um vector, nunca deve ser alterado.

Exemplos:

```
char s[20] = "Ola";

s="ole"     /* Erro de Compilação. Devia usar strcpy     */
s++         /* Erro: Não podemos alterar s               */
s+1         /* OK: Não estamos alterando s               */
*s          /* OK.                                       */
(*s)++      /* OK: Não estamos alterando s, mas sim um dos seus chars */
s = s-2     /* Erro: Não podemos alterar s               */
s > s+1     /* OK.                                       */
s+1-s       /* OK.                                       */
```

Autor: Sílvia António
Ano Académico: 2026/2027

Exercícios

1. De acordo com as declarações abaixo, indique o valor das seguintes expressões
   a) s[2] =?
   b) *(ptr+3)=?
   c) *(s+3)=?
   d) ptr[2] =?

```
char s[] = "planetaazul";
char * ptr = s; //fica com o valor de &s[0];
```

Ponteiros de ponteiros

- Usamos quando desejamos armazenar o valor (endereço) de um ponteiro.

Exemplo:

```
int x = 3;
int * ptr;
int * ptr_ptr;
ptr = &x;
ptr_ptr = &ptr;
```

Passagem de ponteiros para funções

- Sempre que passamos um vector como parâmetro de uma função,

estamos na realidade a passar apenas o endereço do primeiro elemento deste vector.

Passagem de ponteiros para funções - Exemplo 1

```
#include <stdio.h>

void adicionaUm(int* ptr) {
      (*ptr)++;
 }

 int main()
{
   int* p, i = 45;
   p = &i;
   adicionaUm(p);

   printf("%d", i); // 46
   return 0;
}
```

Passagem de ponteiros para funções - Exemplo 2

```
#include <stdio.h>

Int strSize(char * str){

   char * pt = str;   int c=0;
    for(*pt != '\0'){
      pt++;    c++;
   }
   return c;
}

main(){

   char nome[100];
   printf("Insira o nome:");
   gets(nome);
   printf("%d\n", strSize(nome));
}
```

Passagem de ponteiros para funções - Exercícios

1. Declara `int x = 8;` e um ponteiro para `x`. Usa o ponteiro para duplicar o valor de `x` e imprime o resultado.
2. Escreva a função **`void incrementar(int *n)`** que soma 1 ao valor recebido.
3. Implemente uma função que recebe uma string e um caracter e retorna a última ocorrência do caracter na string. Caso não exista, retorna NULL. **char * lastOccur(char * str, char ch);**

BIBLIOGRAFIA

1. Damas. Luis, (2015), Linguagem C, 24ª Ed, FCA



MUITO OBRIGADO PELA ATENÇÃO!